# Laboratorio: Sistema de Auditoría Forense para FAT32-FUSE

## FaMAFyC - Sistemas Operativos 2026

**Fecha de entrega: 4 de Noviembre 23:59**

## Introducción y Objetivos

En este laboratorio van a implementar un **Sistema de Auditoría Forense** para un filesystem FAT32 montado con FUSE. El objetivo es registrar operaciones críticas del filesystem en un log binario y desarrollar un sistema que permita **recuperar archivos borrados**.

La parte central del laboratorio es entender **cuándo un archivo borrado puede recuperarse y cuándo no**. Por ejemplo: si borrás un archivo que usaba ciertos clusters, y luego creás otro archivo que reutiliza esos mismos clusters sobrescribiendo los datos, entonces ya no podés recuperar el primero. El sistema debe detectar automáticamente estas situaciones consultando los logs.

## Conceptos que van a aprender

- Diferencia entre operaciones de alto nivel (syscalls) y cambios de bajo nivel (disk writes)
- VFS como capa de abstracción
- Estructuras de un sistema de archivos: sector de boot, tablas de archivos y sector de datos.
- Mapeo de memoria
- Borrado lógico vs físico
- Serialización de estructuras en formato binario
- Detección de conflictos de recursos (clusters)

## Infraestructura Provista

Les damos implementado:

## 1. Esqueleto funcional del filesystem

El código base tiene todas las operaciones FUSE implementadas: `read`, `write`, `unlink`, `rmdir`, `mkdir`, `rename`, etc. Pueden montar el filesystem y realizar operaciones normalmente.

## 2. Herramientas auxiliares

### `audit_reader` - Lector de logs

Programa en C que lee y muestra el contenido de `/.audit/events.bin` en formato legible.

**Uso:**

```
Shell
./audit_reader /mnt/fuse/.audit/events.bin
```

**Salida ejemplo:**

```
None
ID    Timestamp            Operation  Path
──────────────────────────────────────────────────

1     2026-10-10 14:30:15  write      /file.txt
2     2026-10-10 14:30:20  unlink     /file.txt
3     2026-10-10 14:30:25  mkdir      /dir1

Total events: 3
```

### `restore` - Recuperación de archivos (esqueleto)

Programa en C que permite recuperar archivos borrados. Les damos la estructura básica, ustedes completan la lógica de recuperación en el código FUSE.

**Uso:**

```shell
./restore /mnt/fuse /dir1/file.txt
```

### 3. Headers compartidos

`src/audit.h` contiene las definiciones de estructuras que comparten el código FUSE, `audit_reader` y `restore`. Esto les permite mantener consistencia entre todos los componentes.

## Consigna

### Parte 1: Creación del Sistema de Logs

**Objetivo:** Al montar el filesystem, crear un directorio `/.audit/` que contenga el archivo de logs `events.bin`.

#### Tareas:

1. **Crear el directorio `.audit` al montar:**
    - Crear el archivo `events.bin` dentro de `.audit/`
    - Si ya existen al montar, cargarlos; si no, crearlos
    - Debe persistir entre montajes (al desmontar y volver a montar, debe conservar los logs anteriores)
2. **Decisión de arquitectura:**
    - ¿Dónde van a poner el código relacionado con logging? (¿nuevo archivo `audit.c`? ¿dentro de `fat_fuse_ops.c`?)
    - ¿Qué funciones van a necesitar? (ej: `audit_init()`, `audit_write_event()`, `audit_read_event()`)
    - ¿Cómo van a modularizar el código para mantenerlo limpio?

**Conceptos:** Creación de archivos y directorios. Persistencia de datos. Modularización de código. Punto de entrada al sistema.

## Parte 2: Estructura de Logs y Primer Evento

**Objetivo:** Definir la estructura de datos para representar eventos y loggear las operaciones de **write** solamente.

### Estructura de tamaño fijo

Todos los eventos deben tener el **mismo tamaño** (similar a las dentries que tienen 32 bytes fijos). Esto facilita la lectura/escritura y el acceso aleatorio al archivo.

**Ejemplo de estructura base** (pueden y deben modificarla):

```C/C++
typedef enum {
    OP_WRITE = 0,
    OP_UNLINK = 1,
    OP_MKDIR = 2,
    OP_RMDIR = 3,
} operation_type_t;

typedef struct __attribute__((packed)) {
    uint64_t event_id;       // ID secuencial
    uint64_t timestamp;      // Unix timestamp
    uint32_t operation_type; // Tipo de operación (enum)
    char username[64];       // Usuario que realizó la operación

    // Información específica de la operación
    // Sugerencia: usar union para diferentes tipos
    char filepath[256];      // Path del archivo
    uint64_t offset;         // Para write: offset
    uint64_t size;           // Para write: tamaño

    // Necesitarán guardar más información
    // ¿Qué información exactamente? Piénsenlo...

    // Padding para completar tamaño fijo (ajustar según necesiten)
    uint8_t _padding[512];
```

```
} audit_event_t;
```

Pueden agregar campos según necesiten, incluso aumentar el tamaño de la estructura, pero **siempre manteniendo el mismo tamaño para todos los eventos**

### ⚠️ Consideraciones importantes sobre el tamaño de la estructura

La imagen del filesystem que les damos usa **clusters de 512 bytes**. Esto significa que deben ser cuidadosos con el tamaño de `audit_event_t`:

- **Tamaño recomendado:** Entre 512 bytes (1 cluster) y 1024 bytes (2 clusters)
- **Problema:** Si la estructura es muy grande, cada evento consume múltiples clusters, llenando rápidamente el filesystem
- **Limitación práctica:** Con una estructura de ~512 bytes y `cluster_chain[32]`, solo pueden recuperar archivos de hasta 32 clusters (16KB con clusters de 512B)

**Tareas de diseño:**

1. Calcular el tamaño total de su estructura usando `sizeof(audit_event_t)`
2. Justificar el tamaño elegido en su informe
3. Para la entrega básica, trabajarán con **archivos de prueba pequeños** (< 16KB)

**Para tests más realistas (opcional):**

- Pueden crear localmente una imagen con clusters más grandes (4KB o 8KB)
- Esta imagen NO debe subirse al repositorio (es muy grande)
- Documentar cómo generarla en su README

Tareas:

1. **Definir `audit_event_t`** con los campos que consideren necesarios

2. **Implementar `audit_log_write()`:** función que registra operaciones de escritura
3. **Interceptar y logguear las llamadas a sistema que involucren escritura** Por ejemplo, la operación `write` en `fat_fuse_write()`.
4. **Escribir el evento al archivo `/.audit/events.bin`** en formato binario

**Preguntas para pensar:**

- ¿Qué información necesitan guardar para cada operación?
- ¿Cómo garantizan que todos los eventos tengan el mismo tamaño?
- ¿Cómo manejan el ID secuencial de eventos? (¿variable global? ¿lo leen del archivo? ¿lo calculan en base a alguna constante?)

**Conceptos:** Serialización binaria, estructuras packed, escritura a bajo nivel, operaciones FUSE.

## Parte 3: Verificación con audit_reader

**Objetivo:** Comprobar que el logging funciona correctamente usando `audit_reader`.

### Tareas:

1. Montar el filesystem y realizar operaciones de escritura (`echo`, redirecciones, etc.)
2. Ejecutar `./audit_reader /mnt/fuse/.audit/events.bin`
3. Verificar que los eventos aparecen correctamente
4. **Si modificaron la estructura:** actualizar `audit_reader.c` para mostrar los campos nuevos

**Conceptos:** Debugging con herramientas externas, validación de datos binarios.

## Parte 4: Logging Completo

**Objetivo:** Extender el sistema de logging para **operaciones críticas** del filesystem.

Operaciones a loggear

- `write`
- `unlink`
- `mkdir`
- `rmdir`

En esta etapa del proyecto deben diseñar qué información van a almacenar para poder recuperar un archivo después de `unlink`. ¿Solo necesitan el primer cluster o la cadena completa? ¿Por qué? ¿Necesitan guardar la dentry completa? ¿Para qué? ¿Cómo manejan archivos fragmentados (clusters no contiguos)?

**Limitación importante:** Para la entrega básica, trabajarán con archivos pequeños (≤ 32 clusters = 16KB con clusters de 512B). Si un archivo excede este límite, pueden:

- Opción 1: No loggearlo (documentar esta limitación)
- Opción 2: Loggear parcialmente (primeros 32 clusters, documentar que recuperación será parcial)
- Opción 3 (punto estrella): Implementar una de las estrategias de archivos grandes

Tareas:

1. **Implementar funciones de logging** para cada tipo de operación
2. **Interceptar las operaciones FUSE** correspondientes
3. **Escribir tests** para verificar que el logging funciona correctamente (usar archivos < 16KB)

**Conceptos:** Interceptación de syscalls, testing de sistemas, cadenas de clusters en FAT.

## Parte 5: Recuperación de Archivos Borrados

**Objetivo:** Implementar el comando `restore` que permite recuperar un archivo borrado utilizando la información de los logs.

### El problema

Cuando hacen `rm file.txt`, el filesystem:

1. Marca la dentry como borrada (primer byte = 0xE5)
2. Libera los clusters en la FAT (marca como 0x0 = FREE)
3. **NO sobrescribe los datos** en los clusters

Los datos siguen físicamente en el disco hasta que otro archivo reutilice esos clusters.

Si no tenemos una fuente adicional de datos, la recuperación de archivos borrados solo podrían **asumir** que los clusters eran contiguos (cluster 100 → 101 → 102...) Esto es poco confiable ya que el archivo podría estar fragmentado.

Por otra parte, puede suceder que entre el momento de borrado y recuperación, otra operación haya sobre escrito sobre los mismos clusters de datos. Por lo tanto, recuperar ese archivo lo dejaría con un contenido inconsistente.

Por ello es que el registro de las operaciones que ocurren en el sistema es clave. Con estos logs podemos obtener no solo la cadena exacta de clusters que formaban el archivo, sino que podemos detectar si hubo o no corrupción de datos.

### Comando `restore`

Deben poder invocar de alguna forma la funcionalidad de `restore` del sistema de archivos desde un programa de usuario, al igual que invocan otras operaciones como lectura y escritura a través de syscalls.

Como no podemos implementar nuevas syscalls en nuestro sistema operativo, hay dos alternativas para implementar este comando:

**Opción 1: Usando `ioctl` (recomendado)**

FUSE provee el mecanismo `ioctl` para definir operaciones custom. El programa `restore` usa `ioctl` para comunicarse con el filesystem. Ustedes deben implementar `fat_fuse_ioctl()` para implementar la lógica del comando de restauración.

```C/C++
// En fat_fuse_ops.c
static int fat_fuse_ioctl(const char *path, unsigned int cmd, void *arg, ...);
```

Investiguen qué es `ioctl` y cómo funciona. Les será útil entender cómo se definen comandos ioctl y cómo se pasan argumentos.

**Opción 2: Interceptando escrituras a un archivo especial**

Alternativamente, pueden crear un archivo de control especial (por ejemplo `/.audit/.restore`) y detectar cuando se escribe en él. El contenido escrito sería el path del archivo a restaurar. Esta opción es más simple pero menos elegante que `ioctl`.

**Nota importante:** Hay dos formas de implementar el comando "restore". Una con ioctl y otra interceptando escrituras a un archivo especial. Usar ioctl es la fórmula más adecuada, pero no funciona para todas las arquitecturas sin configuraciones especiales. Es parte del laboratorio decidir cuál de las dos quieren utilizar.

Les recomendamos que primero implementen el comando y una vez que funcione pasen a la implementación del algoritmo.

### Algoritmo de recuperación

```
None
restore /dir1/file.txt:
```

1. Buscar en los logs el registro de la operación de eliminación original
2. Extraer datos relevantes del evento
3. Verificar si los clusters fueron modificados DESPUÉS del unlink:
4. Si no hay conflictos, reconstruir el archivo

Esta consigna está poco definida a propósito. Si encuentran un mejor algoritmo, pueden implementarlo. Tienen muchas posibles soluciones, y su tarea es poder distinguir entre un buen y un mal diseño.

Hay otros aspectos a tener en cuenta en su implementación. Deben elegir qué cantidad de datos almacenar en sus logs. Mientras más información tengan, mejor será la recuperación. Sin embargo, el sistema se volverá más lento y ocupará más espacio en la imagen.

Existen muchos casos borde:

- ¿qué sucede cuando el archivo de logs se vuelve demasiado grande?
- ¿qué hacemos con los logs al desmontar el sistema de archivos?
- ¿qué sucede si queremos restaurar un archivo pero ya no existe su directorio padre? ¿qué sucede si ya hay otro archivo con el mismo nombre?
- ¿Qué pasa si el archivo nunca fue borrado, o no está en los logs?

Hay un balance entre tener una buena cobertura de funcionalidades y mantener un código simple y limpio. Pueden decidir no implementar estos casos bordes, pero deben notificar al usuario y no permitir que el sistema falle silenciosamente.

**Conceptos**: organización de datos y metadatos en un sistema de archivos. Estructura de los sistema de archivos FAT32 y sus limitaciones.

## Evaluación

### Requisitos para aprobar (Partes 1-5):

- ✅ Código compila sin warnings
- ✅ Logging funciona para todas las operaciones solicitadas (write, unlink, mkdir, rmdir)
- ✅ `audit_reader` muestra eventos correctamente
- ✅ `restore` recupera archivos borrados cuando es posible
- ✅ `restore` detecta correctamente cuando NO es posible recuperar (clusters sobrescritos)
- ✅ Tests provistos por la cátedra pasan
- ✅ Commits pequeños y descriptivos (todos los integrantes contribuyen)
- ✅ Código formateado con `clang-format`

### Entregables:

1. **Código funcional** en Bitbucket
2. **Tests propios** (logging + restore exitoso + restore con conflicto)
3. **Informe breve en archivo INFORME.md** en el mismo repositorio (máximo 5 páginas):
    - Decisiones de arquitectura (modularización del código)
    - Qué información guardan para cada tipo de operación y **por qué**
    - Detallar el algoritmo de restauración y detección de colisiones
    - Explicar brevemente las decisiones de diseño
4. **Video** (máximo 7 minutos). Todos los integrantes del grupo deben participar.
    - Demostración del sistema funcionando
    - Mostrar logs con `audit_reader`
    - Demo de `restore` exitoso: borrar archivo, recuperarlo, verificar datos intactos

- Demo de `restore` fallido: borrar archivo, crear otro que use mismo cluster, intentar restore (debe fallar)
- Explicación conceptual de por qué `unlink` puede recuperar datos aunque se "borró" el archivo
- Explicación de las limitaciones de su sistema: en qué casos no puede recuperar la información, en qué casos falla y qué casos borde eligieron no implementar.

## Puntos Estrella

Dependiendo del tiempo disponible, podemos pedirles que implementen algunas de estas funcionalidades más complejas, a su elección.

### ⭐ Restore recursivo de directorios

Implementar `restore -r /dir/` para recuperar un directorio completo con todos sus archivos y subdirectorios.

**Desafíos:**

- Encontrar todos los eventos relacionados con el directorio
- Restaurar en el orden correcto (directorio padre antes que archivos)
- ¿Qué hacer si algunos archivos no se pueden recuperar pero otros sí?
- ¿Implementar dry-run para verificar primero?

### ⭐ Visualizador web interactivo

Crear una interfaz web que muestre los logs en tiempo real con:

- Timeline de eventos
- Filtros por tipo de operación
- Indicadores de qué archivos borrados son recuperables
- Botón para ejecutar `restore` desde la web

Les recomendamos usar Python + Javascript, y utilizar en gran medida IA para el diseño web.

## ⭐ Optimización del allocator

Modificar la asignación de clusters para maximizar recuperabilidad:

- Asignar clusters secuencialmente (watermark)
- Solo reutilizar clusters antiguos cuando no quede espacio
- Mostrar en el informe: comparación de tasa de recuperación con/sin optimización

## ⭐ Recuperación de archivos grandes

La implementación básica está limitada a archivos de hasta 32 clusters (~16KB con clusters de 512B). Implementar soporte para archivos más grandes usando alguna de estas estrategias:

**Opción 1: Optimización para clusters contiguos**

- Detectar si un archivo usa clusters contiguos (no fragmentado)
- Para archivos contiguos, solo guardar `start_cluster` + `length`

**Opción 2: Eventos de continuación**

- Crear un tipo de evento especial `OP_UNLINK_CONTINUATION`
- El evento `OP_UNLINK` principal guarda primeros 32 clusters + flag indicando continuación
- Evento(s) adicionales guardan resto de la cadena
- Al recuperar, buscar y combinar todos los eventos relacionados

**Opción 3: Archivo auxiliar de cadenas**

- Crear `/.audit/chains.bin` separado del log principal
- Los eventos `OP_UNLINK` guardan offset al archivo de cadenas
- Las cadenas largas se almacenan en el archivo auxiliar
- Trade-off: más complejo pero más eficiente en espacio

**Evaluación:**

- Documentar en informe: estrategia elegida y justificación
- Demostrar recuperación de archivo > 16KB
- Comparar overhead de espacio vs. implementación básica

## Recursos y Ayuda

### Funciones útiles del código base:

- `fat_file_pread()` / `fat_file_pwrite()`: Leer/escribir datos de archivos
- `fat_table_get_cluster_value()`: Leer valor de la FAT para un cluster
- `fat_table_set_cluster()`: Modificar valor en la FAT
- `fat_tree_search()`: Buscar un archivo por path

### Debugging:

- Usen `DEBUG()` generosamente (ver `fat_util.h`)
- Monten con `-d` para ver todos los mensajes: `./fat-fuse disk.img /mnt/fuse -d`
- `audit_reader` es útil para ver qué se está loggeando realmente

### Para entender la cadena de clusters:

```shell
# Ver cómo un archivo usa múltiples clusters
# Repite la letra 'A' (con saltos de línea) durante 5,000 líneas
yes A | head -n 5000 > mnt/bigfile.txt
# En el código, hacer DEBUG de la cadena de clusters
# Verán algo como: cluster 100 → 101 → 102 → END
```

## Preguntas Frecuentes

**P: ¿Podemos cambiar la estructura `audit_event_t` que nos dan de ejemplo?** R: Sí, es solo una sugerencia. Pueden agregar/quitar campos según necesiten, pero deben mantener **tamaño fijo**.

**P: ¿Cómo guardamos la cadena de clusters en el log?** R: Pueden usar un array de tamaño fijo (ej: `uint32_t cluster_chain[32]`) y un contador de longitud. Con clusters de 512B, esto limita la recuperación a archivos de hasta 16KB. Para archivos más grandes, ver el punto estrella de "Recuperación de archivos grandes".

**P: ¿Qué pasa si un archivo es más grande que el límite de mi `cluster_chain[]`?** R: Para la entrega básica, tienen dos opciones:

1. Rechazar el logging (retornar error, el archivo se borra pero no queda registrado)
2. Loggear solo los primeros N clusters (archivo se podrá recuperar parcialmente)

Ambas opciones son válidas si las documentan. Para soporte completo, ver punto estrella.

**P: ¿Por qué trabajamos con archivos tan pequeños (16KB)?** R: Es una limitación del tamaño de la estructura de logs vs. el tamaño de los clusters. La imagen del repositorio usa clusters de 512B para mantener el tamaño del archivo pequeño (~10-20MB). Pueden crear localmente una imagen con clusters más grandes para tests más realistas.

**P: ¿Qué pasa si hay múltiples archivos con el mismo nombre borrados?** R: Para la entrega básica, pueden restaurar el más reciente.

**P: ¿Qué pasa si el archivo de logs crece mucho?** R: Por ahora no se preocupen. Para la entrega básica pueden asumir que el log no crecerá indefinidamente. Para evitar que su imagen se llene demasiado de basura, eliminenla y vuelvan a empezar con la imagen limpia que está en el repositorio.

## Cronograma Sugerido (4 semanas)

- **Semana 1:** Partes 1-2 (creación del sistema de logs, estructura de datos, logging de writes)
- **Semana 2:** Partes 3-4 (verificación con audit_reader, logging completo de todas las operaciones)
- **Semana 3:** Parte 5 (implementar restore: búsqueda en logs, detección de conflictos)
- **Semana 4:** Parte 5 (completar restore: reconstrucción), tests finales, informe, video

**¡Éxitos!**